

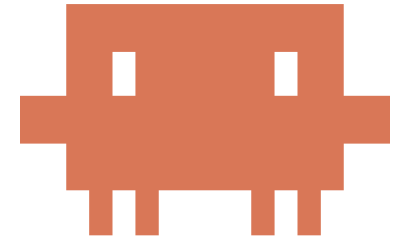
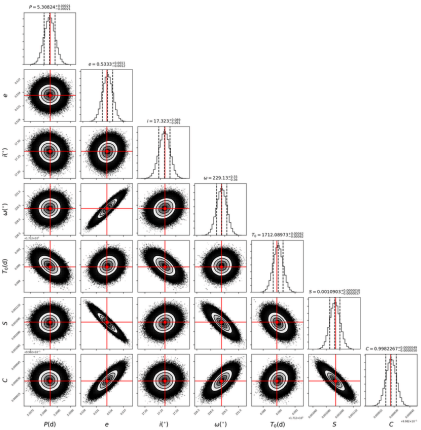
From Supercomputers to AI Agents

Modern workflows for high-performance computing

Tom Kimpson

MACSYS 101 series · September 2026

github.com/tomkimpson/macsys_101_hpc_and_ai_agents



Two skills

Part 1 — Slurm

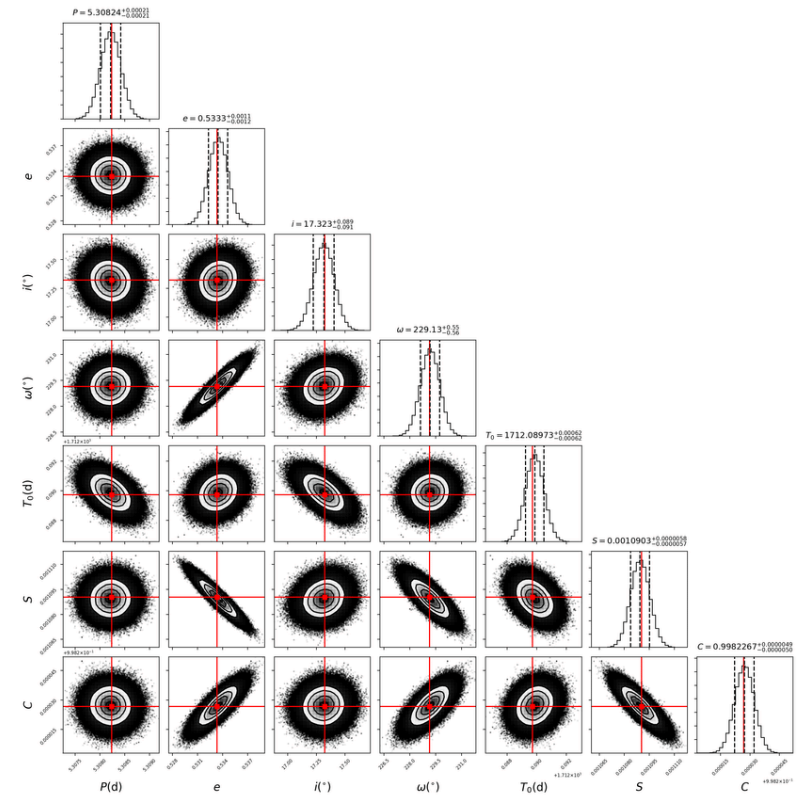
How to run intensive computations on a remote machine

Part 2 — AI agents

How to get an agent to write/debug your code

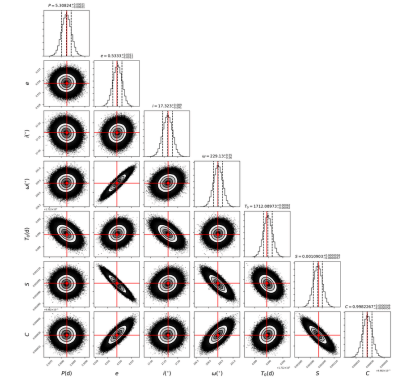
We will be jumping back and forth between these slides and some live terminal demos

Part 1 : HPC + slurm



Why bother?

Lots of science is now compute-intensive and/or scalable



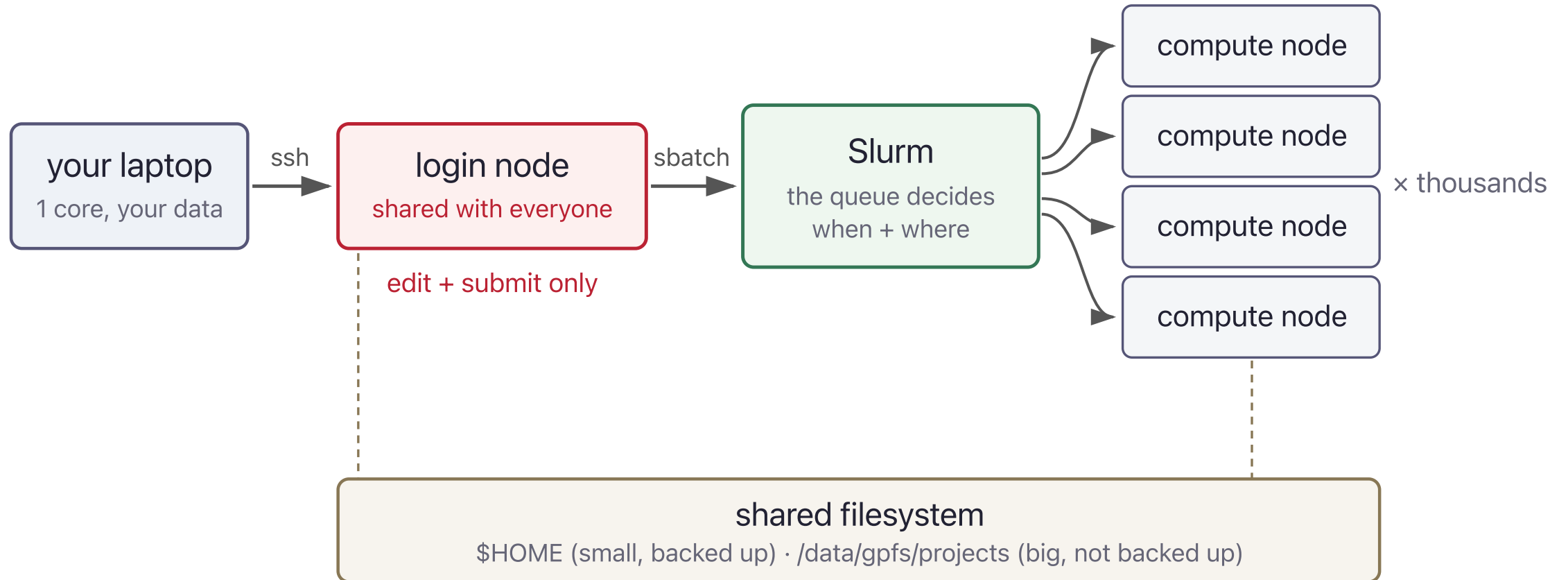
Bottlenecked by your laptop

- Your job needs 200 GB of RAM. Your laptop has 16.
- One run takes 6 hours. You need 10,000 runs.
- You closed the lid / it went to sleep / you needed to fly somewhere.
- Your "quick test" has been running since Tuesday and you can't use your machine.

What you get instead

- Hundreds of independent runs at once — parameter sweeps, MCMC chains, bootstrap replicates, seeds
- Nodes with 100s of cores, TB of RAM, GPUs
- Jobs queue and run while you sleep; you get an email when they're done
- Reproducibility for free: the submission script *is* the record of what you ran

What a cluster actually is



How to get onto the cluster

Before you can connect

- An account — request via your institution's HPC service desk
- Know your hostname:
`spartan.hpc.unimelb.edu.au` (or whichever cluster)
- On/off campus? Some clusters require VPN from outside

Connecting

- **ssh** — the baseline. `ssh user@host`
 - Set up `~/.ssh/config` (or alias) on day one so it's just `ssh spartan`
- **mosh** — ssh that survives suspend, wifi drops, and IP changes
 - Needs mosh installed on the cluster; not all sites allow it
- **VSCode Remote-SSH** — edit files as if local, integrated terminal
 - Caveat: runs a server on the login node, can be resource-hungry. Some sites discourage or block it

How to run a job

You don't run your code directly. Instead you **describe a job** and the scheduler (slurm = Simple Linux Utility for Resource Management) decides when it runs.

Nothing runs on the login node

You tell the scheduler

- How long it will take — `--time`
- How much memory — `--mem`
- How many cores — `--cpus-per-task`
- Which pool of nodes — `--partition`
- Who pays — `--account`

What the scheduler does

- Allocates your job a place in the queue
- Kills you at `--time` .
- Kills you at `--mem` .

*The queue is not first-come-first-served. It is an optimiser: **Ask for less, wait less.***

How to describe a job

To run a job you have to write a shell script. A job script is just a shell script. The `#SBATCH` lines are comments to the shell and instructions to the scheduler.

```
#!/bin/bash
#SBATCH --job-name=ou-fit           # what shows up in queue
#SBATCH --account=oz022            # who gets billed
#SBATCH --partition=skylake        # which pool of nodes
#SBATCH --time=00:05:00           # HH:MM:SS – a hard kill, not a hint
#SBATCH --ntasks=1                # one process...
#SBATCH --cpus-per-task=1          # ...on one core
#SBATCH --mem=2G                   # for the whole job – also a hard kill
#SBATCH --output=logs/%x-%j.out    # %x = job name, %j = job id
#SBATCH --mail-type=END,FAIL       # tell me when it lands, or dies
#SBATCH --mail-user=you@example.edu

module purge                       # start from a known state
module load python-scientific/3.11.3-foss-2023a
source /fred/oz022/${USER}/venvs/macsys/bin/activate

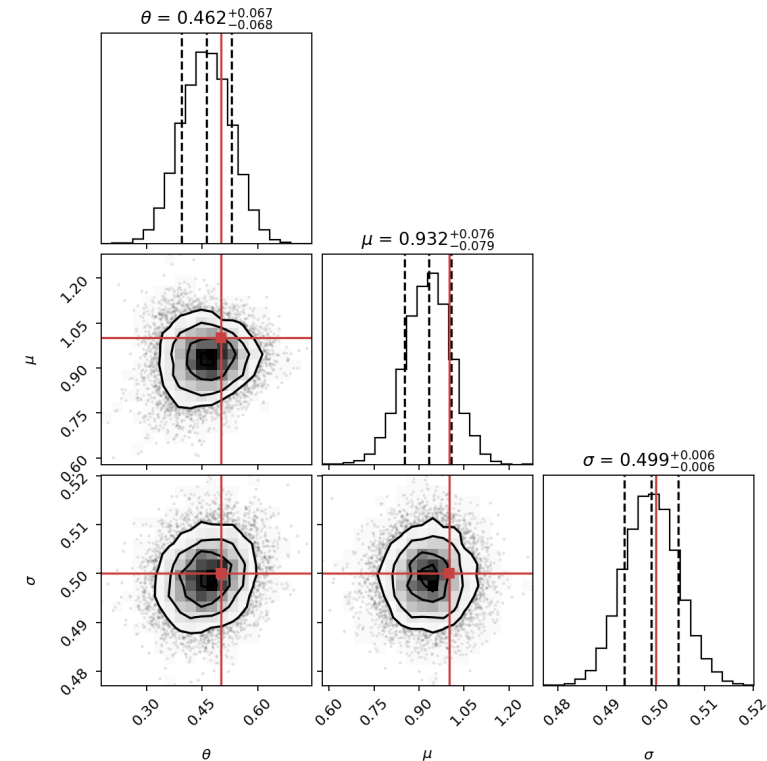
srun python 01_mcmc.py --out posterior.png
```

Basic driving skills

- `sinfo` to see what's available
- `sbatch` submits your job
- `sinteractive` for a shell on a compute node — debug there, never on the login node
- `queue --me` to see your jobs, and why they're waiting
- `scancel $ID` to kill one
- `sacct -j $ID` for what happened to it — `queue` forgets finished jobs
- `seff $ID` for how much you actually used

[Example job monitor from Swinburne](#)

e.g. `sbatch 01_run.sh`



One job → one hundred

An **array job** is the same script run N times.

```
#SBATCH --array=0-99%20      # 100 tasks, 20 at once

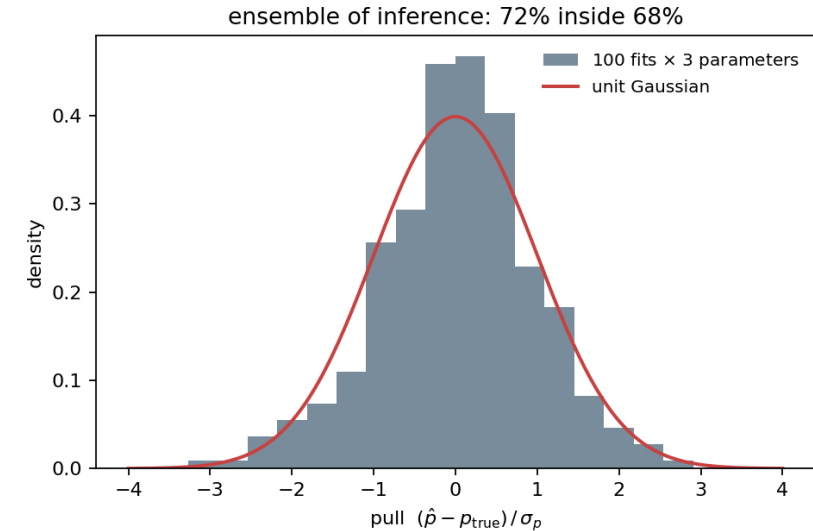
srun python 02_inference.py \
    --task-id ${SLURM_ARRAY_TASK_ID} \
    --out results/fit_${SLURM_ARRAY_TASK_ID}.npz
```

- `--time` and `--mem` are **per task**, not for the array.
- `%20` throttles you. An unthrottled 5,000-task array makes you unpopular!
- Each task writes its own file.

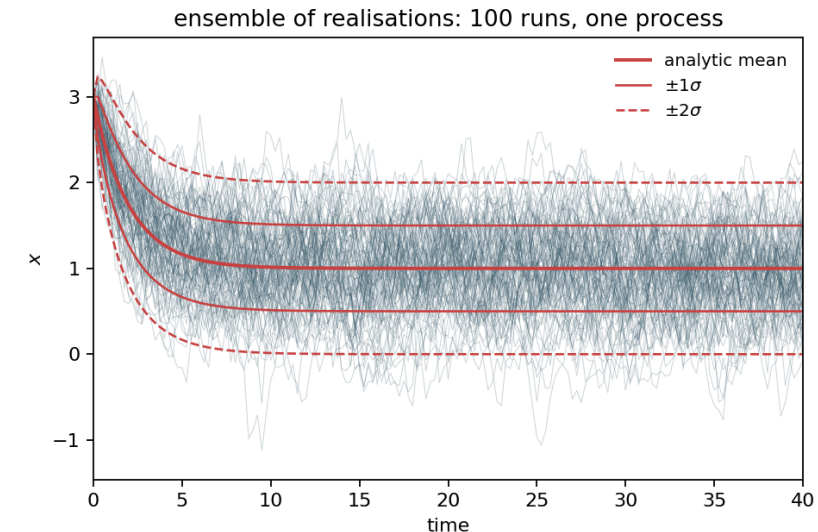
```
sbatch 02_sweep.sh
queue --me
sbatch 02_collate.sh
```

Slurm can hold the collate job until the array finishes — see the appendix.

02_sweep.sh — 100 MCMC fits



03_sweep.sh — 100 forward simulations



Gotchas

	The mistake	The fix
1	Running the real thing on the login node	<code>sinteractive</code> for debugging. That's what it's for.
2	<code>--time</code> too short → job killed at 99%	Time a short version first, then ask for ~1.5×
3	<code>--time</code> / <code>--mem</code> too <i>long</i> → queued for hours	<code>seff</code> after every job. Bring the numbers down.
4	Results written to <code>\$HOME</code>	Small and backed up. Data goes in project storage.
5	Environment differs between login and compute	<code>module purge</code> first, then load explicitly, in the script

Cheat sheet

Getting on and getting oriented

```
ssh spartan.hpc.unimelb.edu.au
module spider python          # search
module load foss/2022a Python/3.10.4
module list                   # what's loaded
sinfo -s                      # partitions, right now
```

Submitting

```
sbatch 01_run.sh
sbatch --parsable 02_sweep.sh      # id only
sbatch --dependency=afterok:$ID 02_collate.sh
sinteractive --time=1:00:00 --mem=8G
```

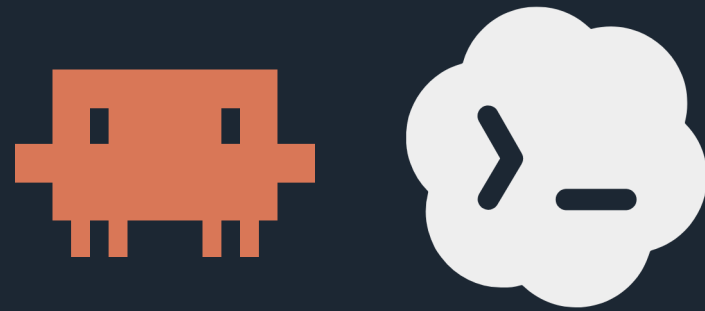
Watching

```
squeue --me
squeue --start -j $ID            # why am I waiting?
scontrol show job $ID
tail -f logs/ou-fit-$ID.out
```

Afterwards

```
seff $ID                        # did I size it right?
sacct -j $ID --format=JobID,State,Elapsed,MaxRSS,ReqMem
scancel $ID                     # one job
scancel ${ID}_7                 # one array task
```

Part 2 : AI agents



How most people use AI already

1. The chat window

ChatGPT / Claude / Gemini in a browser tab.

You paste in the traceback. It pastes back a fix. You paste that into your editor and hope you copied all of it.

2. Tab-complete

Copilot's grey ghost text in VSCode.

It guesses the next line or two from what is above the cursor. You hit `Tab`.

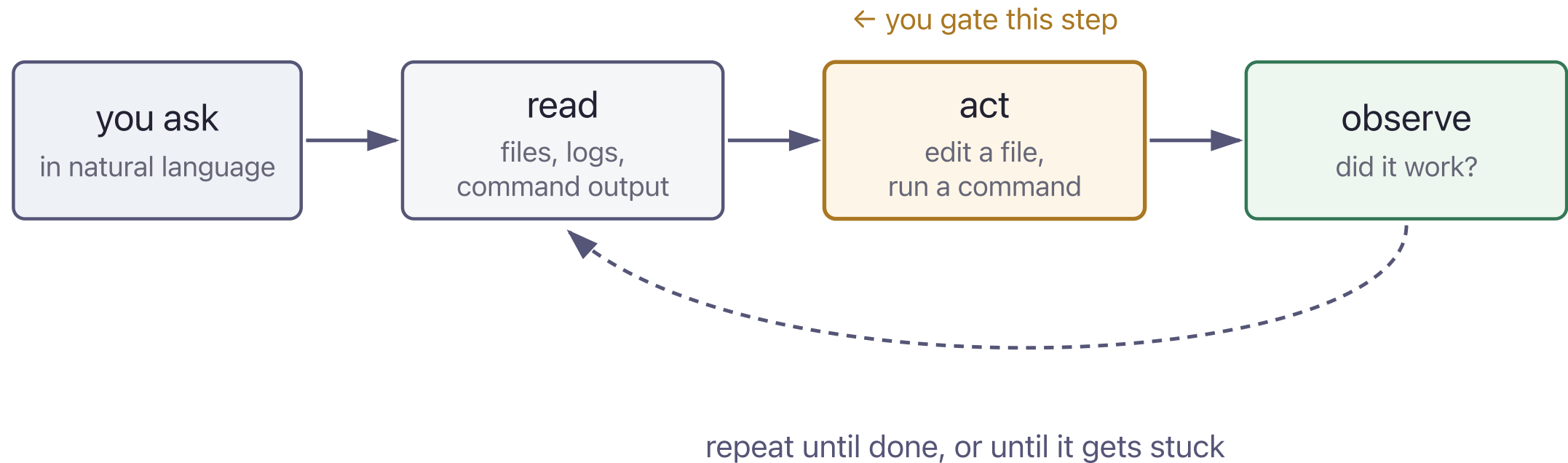
3. Inline edit

Highlight a function, `Cmd-K`, "*vectorise this*".

The same conversation as (1), but it can at least see the file you are in.

AI agent

model + tools + loop



The landscape



Claude Code

Anthropic. Terminal-first, also in the IDE.



Codex

OpenAI. Terminal and cloud.



Antigravity CLI

Google. Terminal only (?).

+ IDE-native agents (Cursor, Windsurf, Copilot agent mode) and open-source harnesses (Aider, OpenHands) — though an open *harness* almost always still calls a closed *model*. On how far the open-weight models actually trail: [How far behind are open models?](#)

The differences are real but small, and they move every few weeks.

Claude Code: basic driving skills

Starting out

<code>/init</code>	Write a starter <code>CLAUDE.md</code> for the repo
<code>/memory</code>	Edit <code>CLAUDE.md</code> afterwards
<code>/permissions</code>	What it may run without asking
<code>/doctor</code>	Setup checkup; also trims a bloated <code>CLAUDE.md</code>

Steering a task

<code>/plan</code>	Plan first, act second. Use before anything large
<code>/diff</code>	What has it actually changed?
<code>/rewind</code>	Roll code and conversation back to a checkpoint

Context is the state

<code>/context</code>	What is filling the window
<code>/compact</code>	Summarise to free space, same conversation
<code>/clear</code>	New conversation, empty context

Before you ship

<code>/simplify</code>	Cleanup pass on what it just wrote: duplication, nesting, naming. Claims to preserve behaviour — so only run it with tests in place
<code>/code-review</code>	Reviews the diff for correctness bugs. <code>--fix</code> applies the findings
<code>/batch</code>	Decomposes a codebase-wide change into 5–30 units, one background agent per unit, each in its own git worktree, each opening a PR

Worth knowing about

`/model` `/effort` · `/usage` (cost) · `/resume` · `/mcp`
· `/btw` (side question, off the record)

Define your own skills

A skill is a folder with a `SKILL.md` in it.

```
---
name: check-refs
description: Verify every citation resolves, and
  supports the claim. Use before submitting.
---

1. Pull every \cite key out of the .tex
2. Cross-check against CrossRef + arXiv
3. Flag anything that does not resolve
```

Then `/check-refs` — or it fires by itself when the description matches.

e.g.

- [google-deepmind/science-skills](#) — genomics, etc.
- [anthropics/skills](#)

demo

1. Lets fix this bug...
2. Lets soup-up our MCMC to use JAX, Hamiltonian Monte Carlo with autodiff, and create a visualisation...
3. Lets vibe code this UI...

Opinions / tips (1/2)

- **Research is not vibe-coding an app.** The litmus test: *would you recognise a wrong answer?*
- **Good software engineering matters more, not less.** Branches, PRs, small commits, unit tests, etc.
- **Keep asks small; `/clear` often.** Hallucinations, unwanted changes, weakened tests and mostly context problems.
- **Parallel agents/tabs work, but easy to lose track.** Start with one.
- **Bio work often banned from using top models !!!**
- **Get model B to review the work of model A** e.g. `/codex:adversarial-review`
- **Write a skill the second time you explain something.** It is a text file, and it is the cheapest thing you can do
- **Delegate straightforward stuff**
 - Plotting and figure iteration
 - File I/O, format wrangling, data munging
 - Test scaffolds around code you already trust
 - Mechanical refactors, porting between languages
 - Reading an unfamiliar codebase — ask before you edit

Opinions / tips (2/2)

Say that you used it

- **Don't hide it.** Say what you used. Everyone else is using it too!
- **Record which models, and for what:** writing, code, algebra, the maths itself.
- **Say what you checked,** and take responsibility for the content.
- **Make every number regenerable.** If each figure and table comes out of a script in the repo, the disclosure writes itself.

The **Leiden Declaration on AI and Mathematics** (June 2026, endorsed by the IMU) recommends a disclosure section leidendeclaration.ai

Declaration on AI usage

Following the disclosure principles of the Leiden Declaration on Artificial Intelligence and Mathematics (Alper et al., 2026), we record the AI tools this work relied on. Large language models (primarily Claude Opus 5, with additional inputs from Claude Fable 5 and GPT-5.6-Sol) were used for writing, refactoring and debugging the benchmark and analysis code. The mathematics was derived without LLM assistance; LLMs were used to check the consistency of the algebra. Every reported number is regenerated from the committed records by scripts in the repository, and the authors take full responsibility for the content of this paper.

*With great power comes great responsibility! **You** are ultimately responsible for the work.*

Controversies & things to be aware of

Don't let it write the paper

- **LLM prose is bad.** Hedged, verbose, dense, awkward flow.
- **Writing is thinking.** See e.g. [Your brain on ChatGPT](#).

Detection is now good

- **Pangram** claims a **1-in-10,000** false-positive rate; an independent [UChicago Booth evaluation](#) found essentially **zero** on long passages
- **Claude [watermarks its own output](#)** (newer models, from Aug 2026 — SynthID-Text, driven by the EU AI Act). Invisible to a reader, detectable with the key.

arXiv will ban you

- **[One year](#)**, from May 2026, for clear evidence of *unchecked* LLM output — hallucinated references, or the chatbot's own meta-comments left in the manuscript.
- Not a ban on using AI — a ban on not reading it. You take **[full responsibility](#)** "*irrespective of how the contents were generated*".
- Fabricated references now appear in **[1 in 277](#)** biomedical papers — a 12x rise since 2023 (*Lancet* audit of 2.5M papers).

Nothing you type is private

- **[Sept 2026](#)**: OpenAI announces a Navier–Stokes result from a ~10,000-agent run. Twelve hours earlier, Buckmaster and Alpöge state that their unpublished Euler work — developed partly inside Codex — had reached OpenAI days before.
- OpenAI denies direct access, but **cannot rule out** their de-identified data having fed model improvement.
- Treat your data / prompts / ideas / discussions accordingly!

What it costs

US\$ / month	Claude	Codex (OpenAI)	Antigravity (Google)
Free	0 — <i>no</i> Claude Code	0 — limited	0 — limited
Entry	"Pro" \$20	"Plus" \$20	"AI Pro" \$20
Heavy — 5x / 20x	"Max" \$100 / \$200	"Pro" \$100 / \$200	"AI Ultra" \$100 / \$200

Subscription — flat rate, metered by a rolling **5-hour** window *and* a **weekly** cap.

API key — per token, no ceiling. Opus 5 is ~\$5 / \$25 per million tokens in / out.

For a group:

- **Claude Team**, 5x Max, \$25 / user.
- **Claude Team for Scientists**, 5x Max, \$15/user. See anthropic.com/news/expanding-support-for-scientists
- UoM - see Kate for a licence
- Others - ask PI to look at Claude Team for Scientists

Consider budgeting for AI usage on any grant applications.

Take-homes

Slurm

- Don't run intensive compute on your laptop
- Nothing runs on the login node.
- HPC doesn't necessarily make your code faster.
- Your resource request is a bid. Ask for less, wait less.
- Try to judge memory/time reqs. accurately
- The submission script is the record of what you ran.

Agents

- **model + tools + loop** — not a chat window. It reads, runs, and checks its own work.
- **Delegate the mechanical stuff** — plots, I/O, refactors, reading unfamiliar code.
- **The litmus test: would you recognise a wrong answer?**
If not, don't delegate it.
- **Context is the state.** Small asks, `/clear` often.
- **Say that you used it.** You own the result.

Thanks

This talk + scripts can be found at

`github.com/tomkimpson/macsys_101_hpc_and_ai_agents`

Spartan docs: dashboard.hpc.unimelb.edu.au · Slurm docs: slurm.schedmd.com · Claude Code: docs.claude.com/claude-code

Appendix

Chaining jobs with `--dependency` appendix

Submit the whole pipeline at once and walk away. Slurm holds each stage until the one before it is done — useful when the array will sit in the queue for six hours.

```
ARRAY=$(sbatch --parsable 02_sweep.sh)          # --parsable prints just "12345679"
sbatch --dependency=afterok:$ARRAY 02_collate.sh
```

Plain `sbatch` prints `Submitted batch job 12345679`; `--parsable` prints the bare id so you can capture it. The collate job queues *immediately*, sits at `PD ... (Dependency)`, and starts by itself when the last task exits cleanly.

flavour	starts when the array...
<code>afterok:\$ID</code>	finished, and every task succeeded
<code>afterany:\$ID</code>	finished, pass or fail
<code>afternotok:\$ID</code>	failed — for cleanup and alerts
<code>aftercorr:\$ID</code>	task <i>i</i> waits for task <i>i</i> , one-to-one
<code>singleton</code>	(one job of this name at a time)

The trap. With `afterok`, one failed task means the dependent job *never runs*. It does not error — it sits at `Reason=(DependencyNeverSatisfied)` until you `scancel` it. Some sites clear it via `kill_invalid_depend`; many do not.

So: `afterok` when a partial result is worthless, `afterany` when your collator can cope with missing files.

FAQs appendix

Why `srun python ...` and not just `python ...` ? `sbatch` allocates resources; `srun` launches a *step* inside that allocation. For a single-core job it barely matters, but it gets you per-step accounting in `sacct`, and it is what you need the moment you go multi-node. Harmless habit, useful later.

`--ntasks` vs `--cpus-per-task` ? `--ntasks` = how many separate processes (MPI ranks). `--cpus-per-task` = how many cores each one gets (threads, OpenMP). One process on 8 threads is `--ntasks=1 --cpus-per-task=8`. Eight MPI ranks is `--ntasks=8 --cpus-per-task=1`. Most of the time you want neither — you want an array of 100 single-core jobs.

What about GPUs? `--partition=gpu-a100 --gres=gpu:1`. Everything else is the same. The hard part is never Slurm; it is whether your code uses the GPU.

What if a few array tasks fail? `sacct -j $ARRAYID --state=FAILED` lists them, and `sbatch --array=7,23,88 02_sweep.sh` reruns just those. This is why one file per task matters.

Running an agent *on* the cluster appendix

You can install and run Claude Code on a login node, which makes the debugging loop tighter — it reads the logs and resubmits without anything round-tripping to your laptop.

Two caveats worth knowing before you do:

- **It is a shared login node.** Agents run commands. Everything on the etiquette slide applies double, and `CLAUDE.md` should say so explicitly.
- **Check your site's policy on outbound API calls** and on what may leave the cluster. If your data is sensitive, the agent reading your files is a data-governance question, not just a technical one.

tmux is useful for agents

- **tmux** — run *on* the cluster. Sessions persist after you disconnect
 - `tmux new -s work`, detach `Ctrl-b d`, come back `tmux a -t work`